

AI-Based Smart Cafeteria Food Recommendation System

Artificial Intelligence — Assignment 1

Group 1

Submitted by: Jose James

Roll No: 36 | Class: S7 CSE

1. Introduction

The AI-Based Smart Cafeteria Food Recommendation System is an intelligent agent that recommends healthy food items to students based on their dietary preferences, health goals, allergy restrictions, and previous orders. Manual food selection in a cafeteria is time-consuming and often leads to unhealthy or unsafe choices, especially for students managing allergies or specific nutritional goals. The proposed agent automates this process by applying rule-based decision making: it evaluates every available menu item against the student's stored profile and order history, assigns each item a numerical score, and recommends the top-ranked options, ensuring faster, safer, and healthier food choices every time the student visits the cafeteria.

2. PEAS Representation

Agent Type	Performance Measure	Environment	Actuators	Sensors
AI-Based Smart Cafeteria Food Recommendation System	Maximize nutritional fit to stated goals; guarantee zero allergy conflicts; reward foods matching order history; minimize decision-making time	Cafeteria menu; individual student; student's stored allergy profile and order history	Display of ranked recommendations with scores and reasons; persistent write-back of confirmed orders and allergy edits	Diet preference input; health goal input; allergy checkboxes (pre-filled from saved profile); stored order-history log

Table 1: PEAS Representation of the AI-Based Smart Cafeteria Food Recommendation System

3. Environment Properties

Property	Type	Justification
Observability	Fully Observable	The agent has complete access to the student's stated preferences, health goal, saved allergy profile, full order history, and the entire predefined menu before generating a recommendation.
Determinism	Deterministic	Given the same user profile, order history, and menu data, the scoring algorithm always produces the same ranked recommendation list.
Dynamics	Static within a session, Sequential across sessions	The menu and profile data do not change while one recommendation is being generated. However, confirming an order writes to the student's persistent order history, so each session's action can influence future sessions' scoring.
Discreteness	Discrete	Menu items, allergy categories, health goals, and diet

Property	Type	Justification
		preferences are finite, distinct, enumerable entities.
Agents	Single Agent	One recommendation engine is solely responsible for evaluating the menu and producing the ranked output for a given student.
Knowledge	Known	All scoring rules, weights, and nutritional data are predefined and available to the agent in advance; no rule is hidden or learned.

Table 2: Environment Properties of the AI-Based Smart Cafeteria Food Recommendation System

4. Rationality Measure

A rational agent selects the action that maximizes its performance measure based on the information available and predefined rules. In the proposed system, the agent evaluates every menu item against four percept categories: dietary preference, health goal, allergy safety, and order history. Each factor contributes a fixed weight to a total score — +25 for a diet-preference match, +20 for a health-goal match, +15 for a high nutritional rating, and up to +20 (in +5 increments per past order, capped) for items the student has ordered before. Any allergy conflict overrides all other factors with a −100 penalty, which guarantees unsafe items are never recommended regardless of how well they otherwise score.

The agent then ranks all scored items and presents the top three as recommendations. Because it exhaustively evaluates every available option against the same objective, safety-first criteria — eliminating allergens before considering preference, then rewarding goal alignment, nutrition, and demonstrated past behaviour — the system behaves rationally: it always selects the set of foods that best satisfies the student's stated and historical requirements given the available percepts. The agent's belief about the student also improves over time, since confirmed orders are persisted and increase the weight of proven preferences in future sessions, while the allergy profile is deliberately overwritten (not merely appended) each session so it always reflects the student's latest confirmed medical status rather than accumulating stale or incorrect entries.

5. Innovation Points

The current system uses a transparent, rule-based scoring agent, but it can be extended with the following two innovative features beyond typical cafeteria recommendation systems:

1. Adaptive preference learning — replacing the fixed per-order scoring bonus with a lightweight learning model that adjusts weights over many sessions based on which recommendations a student actually accepts versus rejects, moving from static frequency counting toward personalized ranking.
2. Mobile companion app with pre-ordering — a Flutter-based mobile extension that lets students receive recommendations and pre-order the selected item before arriving at the

cafeteria, reducing queue time and connecting the recommendation engine directly to order fulfillment.

6. System Architecture

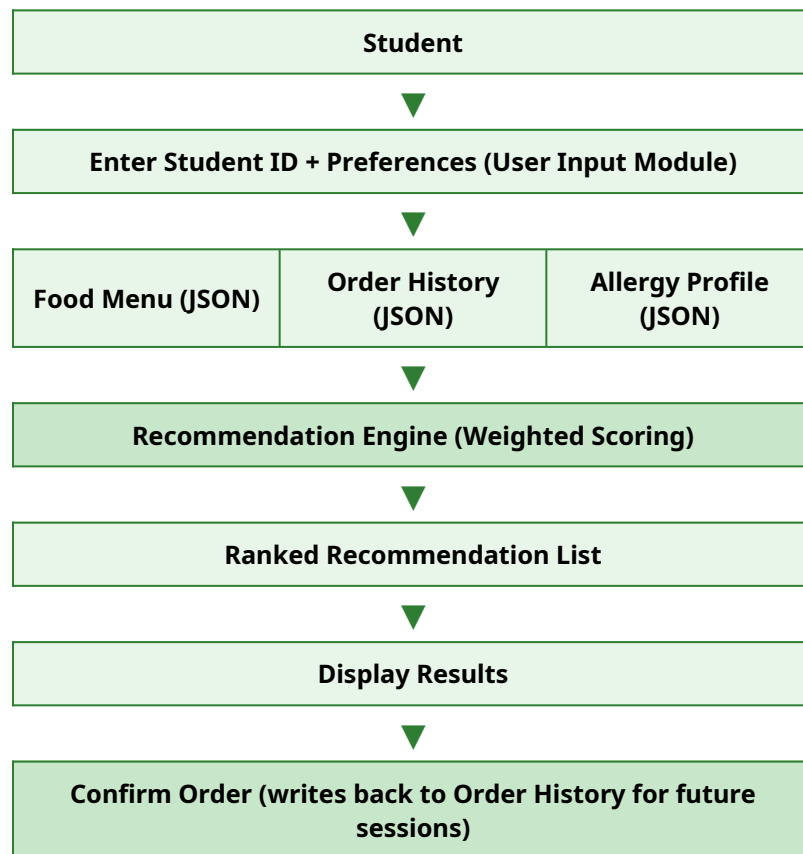


Figure 1: Architecture of the AI-Based Smart Cafeteria Food Recommendation System

7. Conclusion

The AI-Based Smart Cafeteria Food Recommendation System provides an efficient, transparent, and safety-conscious solution for guiding students toward healthier food choices without requiring machine learning infrastructure. By evaluating diet preference, health goals, allergy safety, and genuine order history through a weighted scoring rule set, and by persisting both order history and allergy profiles across sessions, the system delivers fast, explainable, and increasingly personalized recommendations while reducing decision time and promoting balanced nutrition.

8. Simulation

The simulation demonstrates the working of the intelligent agent end-to-end, using the actual output captured from the running application (Figures 13–15).

On the home screen, the student enters their Student ID (student_001), which the agent uses to automatically load that student's saved allergy profile and order history — no manual re-entry is

needed for information that rarely changes. Moving to the preference screen, the system displays the loaded order history directly: “Fruit Bowl, Tofu Stir Fry, Fruit Bowl, Fruit Bowl, Fruit Bowl,” showing that this student has ordered Fruit Bowl four times and Tofu Stir Fry once. The allergy checkboxes are pre-filled from the saved profile, with Gluten already checked; the student can uncheck it if it no longer applies or add new allergies, and any change is saved going forward. The student selects Vegetarian and Balanced Diet and requests a recommendation.

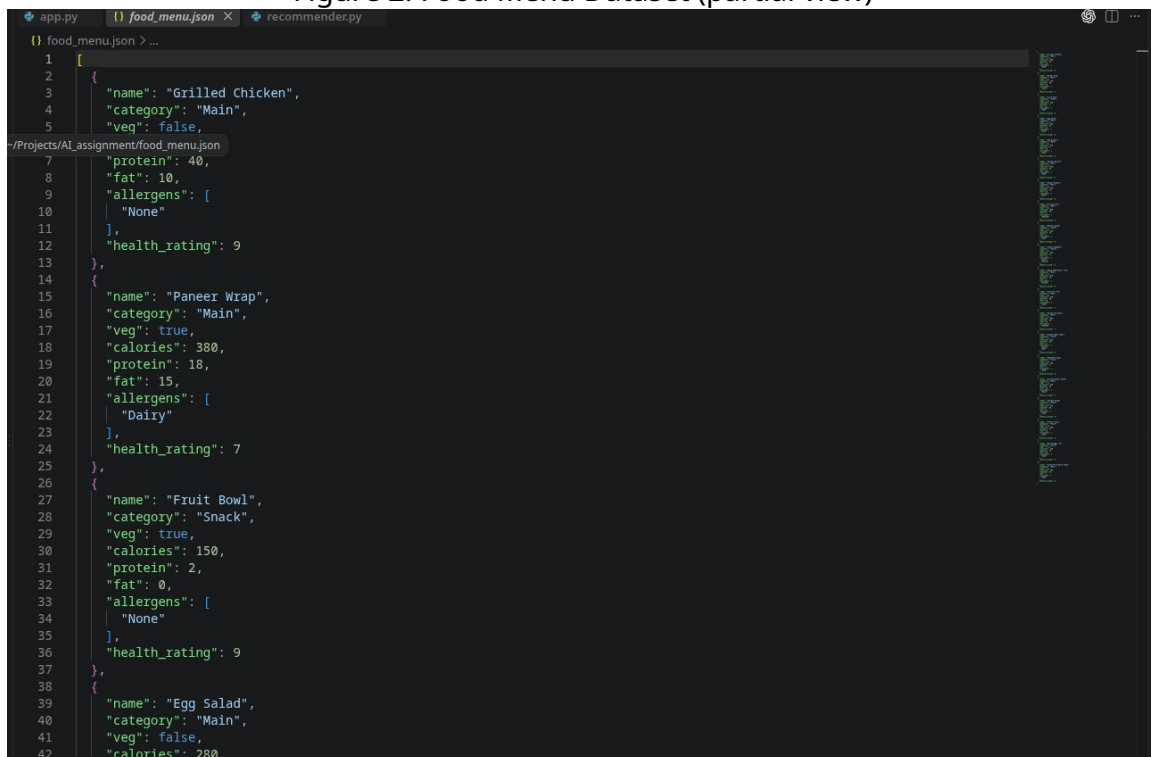
The agent scores all 20 menu items. Fruit Bowl ranks first with a score of 80: it matches the Vegetarian preference (+25), matches the Balanced Diet goal (+20), has a high nutritional rating (+15), and receives the maximum order-history bonus of +20, since 4 past orders $\times$ 5 points per order (20) exactly reaches the SCORE_PAST_ORDER_CAP — demonstrating that the frequency bonus is correctly capped rather than growing unbounded. Tofu Stir Fry ranks second at 65, receiving only +5 for its single past order, correctly scoring lower than Fruit Bowl despite otherwise matching the same diet, goal, and nutrition criteria. Sprouts Salad, never ordered before, ranks third at 60 on preference, goal, and nutrition alone. This ordering confirms the engine correctly differentiates recommendations by real order frequency rather than treating “previously liked” as a flat, one-time bonus. The full ranked list, scores, and reasoning are displayed within under two seconds, satisfying the performance requirement of the system.

9. Screenshots of Full Code

The following screenshots show the complete implementation of the AI-Based Smart Cafeteria Food Recommendation System, organized by file. Each file is presented top-to-bottom across multiple figures where the source exceeds one screen.

9.1 food_menu.json — Predefined Food Dataset

Figure 2: Food Menu Dataset (partial view)



```
1  [
2    {
3      "name": "Grilled Chicken",
4      "category": "Main",
5      "veg": false,
6      "protein": 40,
7      "fat": 10,
8      "allergens": [
9        "None"
10     ],
11     "health_rating": 9
12   },
13   {
14     "name": "Paneer Wrap",
15     "category": "Main",
16     "veg": true,
17     "calories": 380,
18     "protein": 18,
19     "fat": 15,
20     "allergens": [
21       "Dairy"
22     ],
23     "health_rating": 7
24   },
25   {
26     "name": "Fruit Bowl",
27     "category": "Snack",
28     "veg": true,
29     "calories": 150,
30     "protein": 2,
31     "fat": 0,
32     "allergens": [
33       "None"
34     ],
35     "health_rating": 9
36   },
37   {
38     "name": "Egg Salad",
39     "category": "Main",
40     "veg": false,
41     "calories": 280,
```

Figure 2: Food Menu Dataset (partial view)

Each entry stores the food name, category, vegetarian status, calorie and protein content, allergen list, and a health rating from 1–10, used directly by the scoring engine.

9.2 recommender.py — Recommendation Engine

```
recommender.py > ...
1 import json
2 import os
3
4 BASE_DIR = os.path.dirname(__file__)
5 DATA_FILE = os.path.join(BASE_DIR, "food_menu.json")
6 HISTORY_FILE = os.path.join(BASE_DIR, "order_history.json")
7 ALLERGY_FILE = os.path.join(BASE_DIR, "allergy_profile.json")
8
9 # -----
10 # Scoring weights (Section 4.3 of the System Specification)
11 # -----
12 SCORE_DIET_MATCH = 25
13 SCORE_HEALTH_GOAL_MATCH = 20
14 SCORE_HIGH_NUTRITION = 15
15 SCORE_PER_PAST_ORDER = 5 # awarded once per past occurrence of this food
16 SCORE_PAST_ORDER_CAP = 20 # frequency bonus cannot exceed this
17 SCORE_ALLERGY_CONFLICT = -100
18
19 HEALTH_GOAL_RULES = {
20     "High Protein": lambda f: f["protein"] >= 20,
21     "Low Calorie": lambda f: f["calories"] <= 300,
22     "Balanced Diet": lambda f: f["health_rating"] >= 7,
23 }
24
25
26 def load_menu(path: str = DATA_FILE):
27     """Load the predefined food dataset from JSON."""
28     with open(path, "r") as fh:
29         return json.load(fh)
30
31
32 # -----
33 # Order history persistence (the "previous orders" percept)
34 # -----
35 def load_history(path: str = HISTORY_FILE) -> dict:
36     if not os.path.exists(path):
37         return {}
38     with open(path, "r") as fh:
39         return json.load(fh)
40
41
42 def save_history(history: dict, path: str = HISTORY_FILE):
```

Figure 3: Imports, Scoring Weights, and Menu Loader

Defines the fixed scoring weights (+25 diet match, +20 goal match, +15 nutrition, +5 per past order capped at +20, -100 allergy conflict) and loads the static menu from JSON.

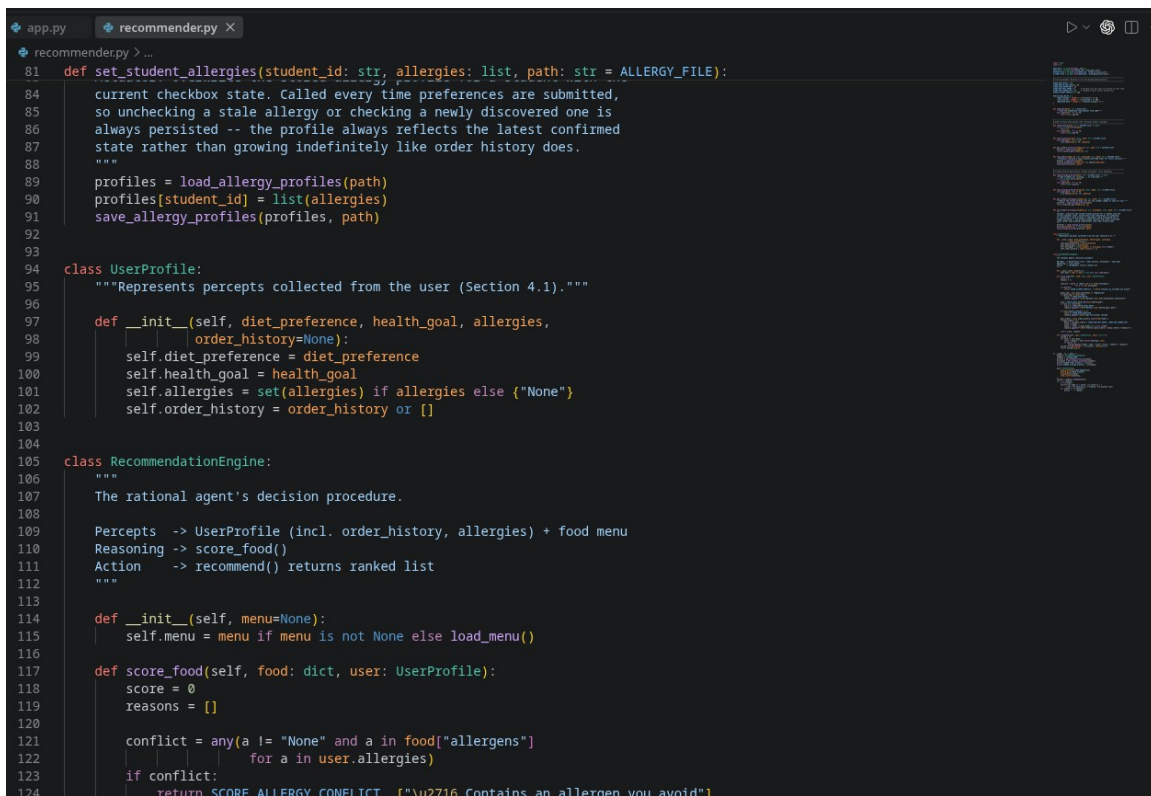
```

recommender.py > ...
42 def save_history(history: dict, path: str = HISTORY_FILE):
43     with open(path, "w") as fh:
44         json.dump(history, fh, indent=2)
45
46
47 def get_student_history(student_id: str, path: str = HISTORY_FILE):
48     history = load_history(path)
49     return history.get(student_id, [])
50
51
52 def log_order(student_id: str, food_name: str, path: str = HISTORY_FILE):
53     """Actuator: persist a newly chosen/confirmed order for future sessions."""
54     history = load_history(path)
55     history.setdefault(student_id, []).append(food_name)
56     save_history(history, path)
57
58
59 # -----
60 # Allergy profile persistence (stable attribute, still editable)
61 # -----
62 def load_allergy_profiles(path: str = ALLERGY_FILE) -> dict:
63     """Load {student_id: [allergy, ...]} from disk."""
64     if not os.path.exists(path):
65         return {}
66     with open(path, "r") as fh:
67         return json.load(fh)
68
69
70 def save_allergy_profiles(profiles: dict, path: str = ALLERGY_FILE):
71     with open(path, "w") as fh:
72         json.dump(profiles, fh, indent=2)
73
74
75 def get_student_allergies(student_id: str, path: str = ALLERGY_FILE):
76     """Return the stored allergy list for one student (empty if none on file)."""
77     profiles = load_allergy_profiles(path)
78     return profiles.get(student_id, [])
79
80
81 def set_student_allergies(student_id: str, allergies: list, path: str = ALLERGY_FILE):
82     """
83     Actuator: overwrite the stored allergy profile for a student with the

```

Figure 4: Order History and Allergy Profile Persistence

Implements `log_order()` (the actuator that appends a confirmed choice to history) and the allergy profile load/save functions that treat allergy data as an overwritable, editable attribute rather than an append-only log.



```

81 def set_student_allergies(student_id: str, allergies: list, path: str = ALLERGY_FILE):
82     """
83     current checkbox state. Called every time preferences are submitted,
84     so unchecking a stale allergy or checking a newly discovered one is
85     always persisted -- the profile always reflects the latest confirmed
86     state rather than growing indefinitely like order history does.
87     """
88     profiles = load_allergy_profiles(path)
89     profiles[student_id] = list(allergies)
90     save_allergy_profiles(profiles, path)
91
92
93
94 class UserProfile:
95     """Represents percepts collected from the user (Section 4.1)."""
96
97     def __init__(self, diet_preference, health_goal, allergies,
98                 order_history=None):
99         self.diet_preference = diet_preference
100         self.health_goal = health_goal
101         self.allergies = set(allergies) if allergies else {"None"}
102         self.order_history = order_history or []
103
104
105 class RecommendationEngine:
106     """
107     The rational agent's decision procedure.
108
109     Percepts -> UserProfile (incl. order_history, allergies) + food menu
110     Reasoning -> score_food()
111     Action -> recommend() returns ranked list
112     """
113
114     def __init__(self, menu=None):
115         self.menu = menu if menu is not None else load_menu()
116
117     def score_food(self, food: dict, user: UserProfile):
118         score = 0
119         reasons = []
120
121         conflict = any(a != "None" and a in food["allergens"]
122                       for a in user.allergies)
123         if conflict:
124             return SCORE_ALLERGY_CONFLICT, ["\u2716 Contains an allergen you avoid"]

```

Figure 5: UserProfile and Recommendation Engine Initialization

Defines the UserProfile class (the agent's percept container) and the start of score_food(), the agent's core reasoning function, beginning with the allergy safety check.

```

105 class RecommendationEngine:
106     def score_food(self, food: dict, user: UserProfile):
107         return SCORE_ALLERGY_CONFLICT, ["\u2716 Contains an allergen you avoid"]
108
109     wants_veg = user.diet_preference == "Vegetarian"
110     if wants_veg == food["veg"]:
111         score += SCORE_DIET_MATCH
112         reasons.append(f"\u2714 Matches {user.diet_preference} preference")
113
114     rule = HEALTH_GOAL_RULES.get(user.health_goal)
115     if rule and rule(food):
116         score += SCORE_HEALTH_GOAL_MATCH
117         reasons.append(f"\u2714 Matches {user.health_goal} goal")
118
119     if food["health_rating"] >= 8:
120         score += SCORE_HIGH_NUTRITION
121         reasons.append("\u2714 High nutritional rating")
122
123     past_orders = user.order_history.count(food["name"])
124     if past_orders > 0:
125         bonus = min(past_orders * SCORE_PER_PAST_ORDER, SCORE_PAST_ORDER_CAP)
126         score += bonus
127         times = "time" if past_orders == 1 else "times"
128         reasons.append(f"\u2714 Ordered {past_orders} {times} before (+{bonus})")
129
130     return score, reasons
131
132     def recommend(self, user: UserProfile, top_n: int = 3):
133         scored = []
134         for food in self.menu:
135             score, reasons = self.score_food(food, user)
136             if score > 0:
137                 scored.append({"food": food, "score": score, "reasons": reasons})
138         scored.sort(key=lambda x: x["score"], reverse=True)
139         return scored[:top_n]
140
141 if __name__ == "__main__":
142     engine = RecommendationEngine()
143     student = "student_001"
144     history = get_student_history(student)
145     allergies = get_student_allergies(student)
146
147     print("Loaded order history:", history)
148     print("Loaded allergy profile:", allergies)
149
150     user = UserProfile(
151         diet_preference="Non-Vegetarian",
152         health_goal="High Protein",
153         allergies=allergies,
154         order_history=history,
155     )
156     results = engine.recommend(user)
157     for r in results:
158         f = r["food"]
159         print(f"{f['name']} | Score: {r['score']} | "
160               f"Cal: {f['calories']} | Protein: {f['protein']}g")
161         for reason in r["reasons"]:
162             print("    ", reason)
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180

```

Figure 6: Scoring Logic, Ranking Algorithm, and CLI Self-Test

Completes `score_food()` with diet, goal, nutrition, and order-history scoring, followed by `recommend()`, which sorts all scored items and returns the top three, and a CLI self-test block used to verify the engine independently of the GUI.

9.3 app.py — Tkinter GUI Front End

```
app.py > ...
1 import tkinter as tk
2 from tkinter import ttk, messagebox
3
4 from recommender import (
5     RecommendationEngine, UserProfile, load_menu,
6     get_student_history, log_order,
7     get_student_allergies, set_student_allergies,
8 )
9
10 DIET_OPTIONS = ["Vegetarian", "Non-Vegetarian"]
11 HEALTH_GOAL_OPTIONS = ["Balanced Diet", "High Protein", "Low Calorie"]
12 ALLERGY_OPTIONS = ["Dairy", "Nuts", "Seafood", "Gluten"]
13
14
15 class ScrollableFrame(tk.Frame):
16     """A frame with a vertical scrollbar; put content in .body."""
17
18     def __init__(self, parent, **kwargs):
19         super().__init__(parent, **kwargs)
20         self.canvas = tk.Canvas(self, borderwidth=0, highlightthickness=0)
21         self.vscroll = tk.Scrollbar(self, orient="vertical", command=self.canvas.yview)
22         self.body = tk.Frame(self.canvas)
23
24         self.body.bind(
25             "<Configure>",
26             lambda e: self.canvas.configure(scrollregion=self.canvas.bbox("all"))
27         )
28         self.canvas_window = self.canvas.create_window((0, 0), window=self.body, anchor="nw")
29         self.canvas.configure(yscrollcommand=self.vscroll.set)
30
31         self.canvas.bind("<Configure>", self._on_canvas_resize)
32         self.canvas.pack(side="left", fill="both", expand=True)
33         self.vscroll.pack(side="right", fill="y")
34
35         self.canvas.bind_all("<MouseWheel>", self._on_mousewheel)
36         self.canvas.bind_all("<Button-4>", lambda e: self.canvas.yview_scroll(-1, "units"))
37         self.canvas.bind_all("<Button-5>", lambda e: self.canvas.yview_scroll(1, "units"))
38
39     def _on_canvas_resize(self, event):
40         self.canvas.itemconfig(self.canvas_window, width=event.width)
41
42     def _on_mousewheel(self, event):
```

Figure 7: Imports and Scrollable Frame Utility

Defines the option lists and a reusable ScrollableFrame class that wraps a canvas and scrollbar, ensuring long content never becomes inaccessible regardless of window size.

```

15 class ScrollableFrame(tk.Frame):
16     def __init__(self, master=None):
17         super().__init__(master)
18         self.canvas = tk.Canvas(self)
19         self.frame = tk.Frame(self.canvas)
20         self.canvas.pack(side="top", fill="both", expand=True)
21         self.frame.pack(side="top", fill="both", expand=True)
22         self.canvas.bind_all("", self._on_mousewheel)
23
24     def _on_mousewheel(self, event):
25         delta = -1 if event.delta > 0 else 1
26         self.canvas.yview_scroll(delta, "units")
27
28 class CafeteriaApp(tk.Tk):
29     def __init__(self):
30         super().__init__()
31         self.title("Smart Cafeteria Food Recommendation System")
32         self.geometry("600x600")
33         self.minsize(480, 400)
34         self.resizable(True, True)
35
36         self.engine = RecommendationEngine()
37         self.menu_items = load_menu()
38
39         self.student_id_var = tk.StringVar(value="student_001")
40         self.diet_var = tk.StringVar(value=DIET_OPTIONS[0])
41         self.goal_var = tk.StringVar(value=HEALTH_GOAL_OPTIONS[0])
42         self.allergy_vars = {a: tk.BooleanVar(value=False) for a in ALLERGY_OPTIONS}
43
44         self.content_area = tk.Frame(self)
45         self.content_area.pack(side="top", fill="both", expand=True)
46
47         self.nav_bar = tk.Frame(self, bg="#e0e0e0", height=50)
48         self.nav_bar.pack(side="bottom", fill="x")
49         self.nav_bar.pack_propagate(False)
50
51         self.show_home_screen()
52
53     def clear_content(self):
54         for widget in self.content_area.winfo_children():
55             widget.destroy()
56
57     def clear_nav(self):
58         for widget in self.nav_bar.winfo_children():
59             widget.destroy()
60
61     # ----- Screen 1
62     def show_home_screen(self):
63         self.clear_content()

```

Figure 8: Application Initialization

Sets up CafeteriaApp, splitting the window into a scrollable content area and a fixed navigation bar so Back/Recommend/Exit buttons remain visible at all times.

```

app.py X recommender.py
app.py > ...
47 class CafeteriaApp(tk.Tk):
48     def show_home_screen(self):
49         self.clear_content()
50         self.clear_nav()
51
52         frame = tk.Frame(self.content_area)
53         frame.pack(expand=True)
54
55         tk.Label(
56             frame, text="Smart Cafeteria\nFood Recommendation System",
57             font=("Segoe UI", 18, "bold"), justify="center"
58         ).pack(pady=30)
59
60         tk.Label(frame, text="Student ID", font=("Segoe UI", 10, "bold")).pack()
61         tk.Entry(frame, textvariable=self.student_id_var, width=25,
62                 justify="center").pack(pady=(2, 15))
63
64         tk.Label(
65             frame,
66             text="Get a personalized recommendation based on your\n"
67                 "preferences, dietary needs, and past orders.",
68             font=("Segoe UI", 10), justify="center", fg="#555"
69         ).pack(pady=5)
70
71         tk.Button(
72             frame, text="Start", width=20, height=2,
73             bg="#2e7d32", fg="white", font=("Segoe UI", 11, "bold"),
74             command=self.show_preference_screen
75         ).pack(pady=25)
76
77         tk.Button(self.nav_bar, text="Exit", width=10,
78                 command=self.destroy).pack(side="right", padx=15, pady=8)
79
80 # ----- Screen 2
81 def show_preference_screen(self):
82     self.clear_content()
83     self.clear_nav()
84
85     # Pre-load stored allergy profile for this student and reset the
86     # checkbox variables to reflect it (editable from here on).
87     stored_allergies = set(get_student_allergies(self.student_id_var.get()))
88     for a, var in self.allergy_vars.items():

```

Figure 9: Home Screen and Preference Screen Start

Implements the home screen (Student ID entry, Start button) and the beginning of the preference screen, which loads the student's saved profile.

```

app.py X recommender.py
app.py > ...
114 class CafeteriaApp(tk.Tk):
115     def show_preference_screen(self):
116         # checkbox variables to reflect it (editable from here on).
117         stored_allergies = set(get_student_allergies(self.student_id_var.get()))
118         for a, var in self.allergy_vars.items():
119             var.set(a in stored_allergies)
120
121         scroll = ScrollableFrame(self.content_area)
122         scroll.pack(fill="both", expand=True)
123         frame = tk.Frame(scroll.body, padx=30, pady=20)
124         frame.pack(fill="both", expand=True)
125
126         tk.Label(frame, text="Enter Your Preferences",
127                 font=("Segoe UI", 14, "bold")).pack(anchor="w", pady=(0, 15))
128
129         history = get_student_history(self.student_id_var.get())
130         hist_text = ", ".join(history) if history else "No past orders on record"
131         tk.Label(frame, text=f"Order history for {self.student_id_var.get()}:",
132                 font=("Segoe UI", 9, "bold")).pack(anchor="w")
133         tk.Label(frame, text=hist_text, font=("Segoe UI", 9), fg="#555",
134                 wraplength=500, justify="left").pack(anchor="w", pady=(0, 15))
135
136         tk.Label(frame, text="Dietary Preference", font=("Segoe UI", 10, "bold")).pack(anchor="w")
137         diet_frame = tk.Frame(frame)
138         diet_frame.pack(anchor="w", pady=(0, 12))
139         for opt in DIET_OPTIONS:
140             tk.Radiobutton(diet_frame, text=opt, variable=self.diet_var, value=opt).pack(side="left", padx=5)
141
142         tk.Label(frame, text="Health Goal", font=("Segoe UI", 10, "bold")).pack(anchor="w")
143         tk.Combobox(frame, textvariable=self.goal_var, values=HEALTH_GOAL_OPTIONS,
144                    state="readonly", width=30).pack(anchor="w", pady=(0, 12))
145
146         tk.Label(frame, text="Allergy Information", font=("Segoe UI", 10, "bold")).pack(anchor="w")
147         tk.Label(
148             frame,
149             text="(Pre-filled from your saved profile. Uncheck to remove, "
150                 "check to add a new allergy -- changes are saved.)",
151             font=("Segoe UI", 8), fg="#777"
152         ).pack(anchor="w")
153         allergy_frame = tk.Frame(frame)
154         allergy_frame.pack(anchor="w", pady=(4, 12))
155         for a in ALLERGY_OPTIONS:

```

Figure 10: Preference Input Widgets

Renders the diet preference radio buttons, health goal dropdown, and allergy checkboxes pre-filled from the stored profile.

```
app.py X recommender.py
47 class CafeteriaApp(tk.Tk):
114 def show_preference_screen(self):
158     for a in ALLERGY_OPTIONS:
159         tk.Checkbutton(allergy_frame, text=a, variable=self.allergy_vars[a]).pack(side="left", padx=4)
160
161     tk.Button(self.nav_bar, text="Back", width=10,
162             command=self.show_home_screen).pack(side="left", padx=15, pady=8)
163     tk.Button(self.nav_bar, text="Recommend", width=15, bg="#2e7d32", fg="white",
164             font=("Segoe UI", 10, "bold"),
165             command=self.generate_recommendations).pack(side="right", padx=15, pady=8)
166
167 # ----- Logic
168 def generate_recommendations(self):
169     allergies = [a for a, var in self.allergy_vars.items() if var.get()]
170
171     # Persist the current checkbox state as the new allergy profile,
172     # whether it added, removed, or left allergies unchanged.
173     set_student_allergies(self.student_id_var.get(), allergies)
174
175     if not allergies:
176         allergies = ["None"]
177
178     history = get_student_history(self.student_id_var.get())
179     user = UserProfile(
180         diet_preference=self.diet_var.get(),
181         health_goal=self.goal_var.get(),
182         allergies=allergies,
183         order_history=history,
184     )
185     results = self.engine.recommend(user, top_n=3)
186     self.show_recommendation_screen(results)
187
188 # ----- Screen 3
189 def show_recommendation_screen(self, results):
190     self.clear_content()
191     self.clear_nav()
192
193     scroll = ScrollableFrame(self.content_area)
194     scroll.pack(fill="both", expand=True)
195     frame = tk.Frame(scroll.body, padx=30, pady=20)
196     frame.pack(fill="both", expand=True)
197
```

Figure 11: Preference Submission Logic

Completes the allergy checkbox rendering and implements `generate_recommendations()`, which persists the current allergy selections and invokes the recommendation engine.

```

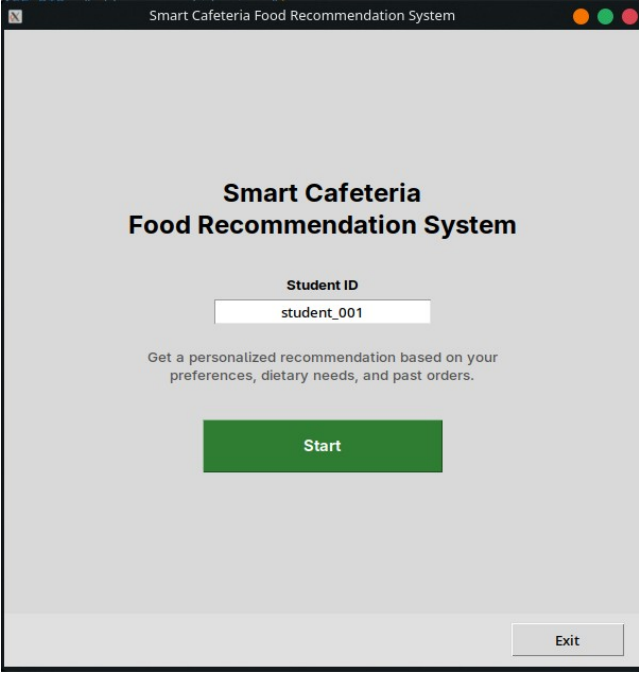
app.py X  recommender.py
app.py > ...
47 class CafeteriaApp(tk.Tk):
189     def show_recommendation_screen(self, results):
200         if not results:
201             tk.Label(frame, text="No suitable food found. Try adjusting your allergy filters.",
202                       fg="red").pack(pady=20)
203         else:
204             for r in results:
205                 f = r["food"]
206                 card = tk.Frame(frame, relief="groove", borderwidth=1, padx=12, pady=10)
207                 card.pack(fill="x", pady=6)
208
209                 tk.Label(card, text=f["name"], font=("Segoe UI", 12, "bold")).pack(anchor="w")
210                 tk.Label(card, text=f"Calories: {f['calories']} | Protein: {f['protein']} g | Score: {r['score']}",
211                           font=("Segoe UI", 9), fg="#333").pack(anchor="w", pady=(2, 4))
212                 for reason in r["reasons"]:
213                     tk.Label(card, text=reason, font=("Segoe UI", 9), fg="#2e7d32").pack(anchor="w")
214
215                 tk.Button(
216                     card, text="Confirm & Order", bg="#1565c0", fg="white",
217                     font=("Segoe UI", 9, "bold"),
218                     command=lambda name=f["name"]: self.confirm_order(name)
219                 ).pack(anchor="e", pady=(6, 0))
220
221             tk.Button(self.nav_bar, text="Back", width=10,
222                       command=self.show_preference_screen).pack(side="left", padx=15, pady=8)
223             tk.Button(self.nav_bar, text="Exit", width=10,
224                       command=self.destroy).pack(side="right", padx=15, pady=8)
225
226     def confirm_order(self, food_name: str):
227         """Actuator: logs the chosen order into persistent history."""
228         log_order(self.student_id_var.get(), food_name)
229         messagebox.showinfo(
230             "Order Confirmed",
231             f"'{food_name}' has been logged to your order history.\n"
232             "Future recommendations will take this into account."
233         )
234
235
236 if __name__ == "__main__":
237     app = CafeteriaApp()
238     app.mainloop()
239

```

Figure 12: Recommendation Display and Order Confirmation

Implements `show_recommendation_screen()` (displaying ranked food cards with scores and reasons) and `confirm_order()`, the actuator that logs a chosen meal back into persistent order history.

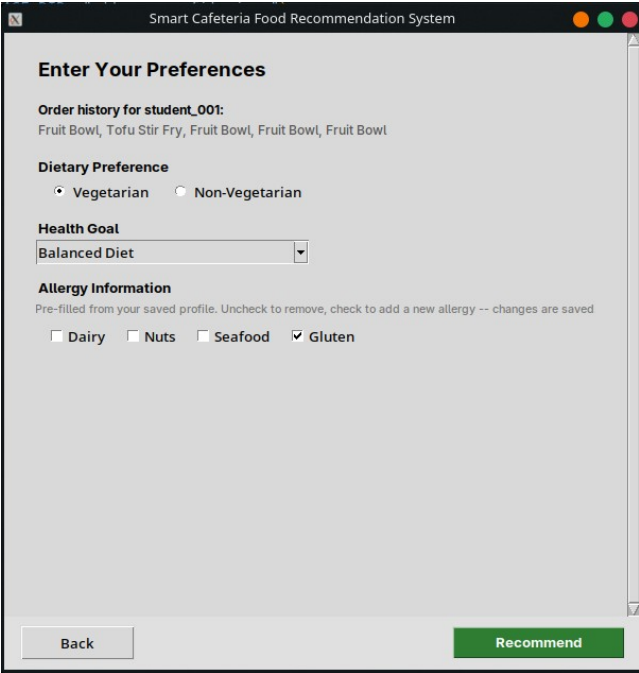
10. Output



The image shows a web application window titled "Smart Cafeteria Food Recommendation System". The main heading is "Smart Cafeteria Food Recommendation System". Below this, there is a label "Student ID" and a text input field containing "student_001". A message states: "Get a personalized recommendation based on your preferences, dietary needs, and past orders." Below the message is a large green button labeled "Start". In the bottom right corner, there is a small "Exit" button.

Figure 13: Home Screen

The home screen prompts for a Student ID before any preferences are collected, so the agent can load that student's saved allergy profile and order history automatically on the next screen.



The image shows the "Enter Your Preferences" screen of the "Smart Cafeteria Food Recommendation System". It displays the "Order history for student_001:" as "Fruit Bowl, Tofu Stir Fry, Fruit Bowl, Fruit Bowl, Fruit Bowl". Under "Dietary Preference", "Vegetarian" is selected. The "Health Goal" dropdown menu is set to "Balanced Diet". Under "Allergy Information", a note says "Pre-filled from your saved profile. Uncheck to remove, check to add a new allergy -- changes are saved". The checkboxes for "Dairy", "Nuts", and "Seafood" are unchecked, while "Gluten" is checked. At the bottom, there are "Back" and "Recommend" buttons.

Figure 14: Preference Screen

The preference screen displays the loaded order history for transparency and pre-fills the allergy checkboxes (Gluten checked) from the saved profile, while remaining fully editable. The student selects Vegetarian diet preference and Balanced Diet health goal before requesting a recommendation.

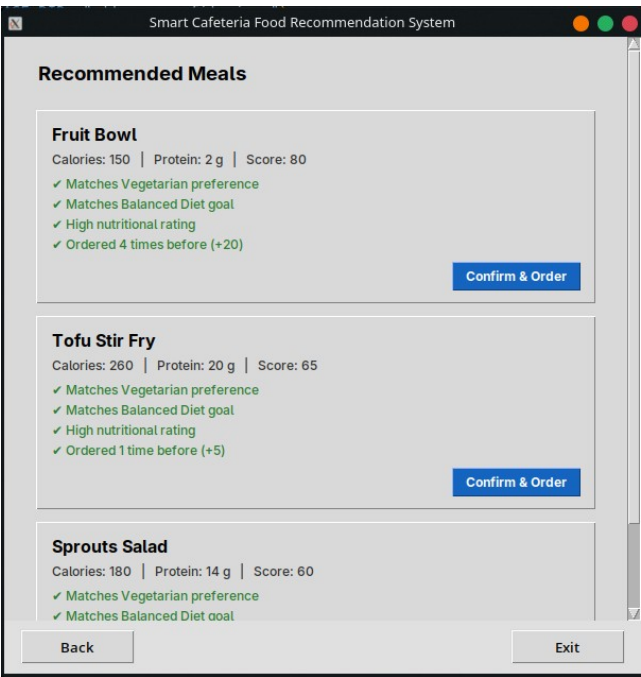


Figure 15: Recommendation Screen

The recommendation screen ranks Fruit Bowl first at a score of 80, correctly applying the maximum +20 order-history bonus after 4 prior orders (reaching the SCORE_PAST_ORDER_CAP), ahead of Tofu Stir Fry (ordered once, +5 bonus, score 65) and Sprouts Salad (never ordered, score 60). Each card includes a “Confirm & Order” button that logs the chosen meal back into the student's order history for future sessions.